

ABSTRACT CLASSES

Aim:

To understand and implement inheritance concepts in Java.

PRE LAB EXERCISE

QUESTIONS

- ✓ What is an abstract class?
 - An abstract class is declared using the abstract keyword in Java.
 - It cannot be instantiated directly using the new keyword.
 - It may contain abstract methods as well as concrete (implemented) methods.
 - It is mainly used as a base class to provide common features for subclasses.

- ✓ Why are abstract methods used?
 - Abstract methods are declared without implementation in the parent class.
 - They force the subclasses to provide their own implementation of the method.
 - This helps in achieving abstraction by hiding implementation details.
 - They ensure that all derived classes follow a common method structure.

- ✓ Difference between abstract class and interface.

Abstract Class	Interface
Uses the keyword <code>abstract</code> .	Uses the keyword <code>interface</code> .
Can contain both abstract and concrete (implemented) methods.	Methods are abstract by default (till Java 7).
Can have instance variables.	Variables are public, static, and final by default.
A class can extend only one abstract class.	A class can implement multiple interfaces.
Can have constructors.	Cannot have constructors.
Provides partial abstraction.	Provides complete abstraction.

IN LAB EXERCISE

Objective:

To implement abstract class and demonstrate abstraction.

PROGRAMS:**1.University System****Scenario:**

A university has different types of courses: Online, Offline, and Hybrid. Each course has a getDetails() method.

Question:

Create an abstract class Course with abstract method getDetails(). Implement OnlineCourse, OfflineCourse, and HybridCourse classes.

Code:

```
abstract class Course {  
    abstract void getDetails();  
}
```

```
class OnlineCourse extends Course {  
    void getDetails() {  
        System.out.println("Online Course: Attend via Internet");  
    }  
}
```

```
class OfflineCourse extends Course {  
    void getDetails() {  
        System.out.println("Offline Course: Attend in classroom");  
    }  
}
```

```
class HybridCourse extends Course {  
    void getDetails() {  
        System.out.println("Hybrid Course: Combination of online and offline");  
    }  
}
```

```
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Course c1 = new OnlineCourse();  
        Course c2 = new OfflineCourse();  
        Course c3 = new HybridCourse();  
  
        c1.getDetails();  
        c2.getDetails();  
        c3.getDetails();  
    }  
}
```

Output:

Online Course: Attend via Internet

Offline Course: Attend in classroom

Hybrid Course: Combination of online and offline

```

2  abstract class Course {
3      abstract void getDetails();
4  }
5  class OnlineCourse extends Course {
6      void getDetails() {
7          System.out.println("Online Course: Attend via Internet");
8      }
9  }
10
11 class OfflineCourse extends Course {
12     void getDetails() {
13         System.out.println("Offline Course: Attend in classroom");
14     }
15 }
16
17 class HybridCourse extends Course {
18     void getDetails() {
19         System.out.println("Hybrid Course: Combination of online and offline");
20     }
21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         Course c1 = new OnlineCourse();
26         Course c2 = new OfflineCourse();
27         Course c3 = new HybridCourse();
28
29         c1.getDetails();
30         c2.getDetails();
31         c3.getDetails();
32     }
33 }

```

```

Online Course: Attend via Internet
Offline Course: Attend in classroom
Hybrid Course: Combination of online and offline

```

2. Employee Payroll System

Scenario:

A company has different types of employees — Regular and Contract. All employees have a salary, but the calculation differs for each type.

Question:

Design an abstract class Employee with an abstract method calculateSalary(). Implement subclasses RegularEmployee and ContractEmployee to calculate salary differently.

Code:

```

import java.util.Scanner;

abstract class Employee {
    String name;
    double baseSalary;

    // Abstract method to calculate total salary
    abstract void calculateSalary();
}

```

```
}
```

```
class RegularEmployee extends Employee {
```

```
    double bonusRate = 0.1; // 10% bonus
```

```
    void calculateSalary() {
```

```
        double totalSalary = baseSalary + (baseSalary * bonusRate);
```

```
        System.out.println("Regular Employee: " + name);
```

```
        System.out.println("Base Salary: " + baseSalary);
```

```
        System.out.println("Total Salary (with 10% bonus): " + totalSalary);
```

```
    }
```

```
}
```

```
class ContractEmployee extends Employee {
```

```
    void calculateSalary() {
```

```
        System.out.println("Contract Employee: " + name);
```

```
        System.out.println("Total Salary: " + baseSalary);
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        // Input for Regular Employee
```

```
        System.out.print("Enter Regular Employee Name: ");
```

```
        String regName = sc.nextLine();
```

```
        System.out.print("Enter Base Salary: ");
```

```
double regSalary = sc.nextDouble();
sc.nextLine(); // Consume newline

// Input for Contract Employee
System.out.print("Enter Contract Employee Name: ");
String conName = sc.nextLine();
System.out.print("Enter Base Salary: ");
double conSalary = sc.nextDouble();

// Create objects
Employee e1 = new RegularEmployee();
e1.name = regName;
e1.baseSalary = regSalary;

Employee e2 = new ContractEmployee();
e2.name = conName;
e2.baseSalary = conSalary;

System.out.println("\n--- Salary Details ---");
e1.calculateSalary();
System.out.println();
e2.calculateSalary();

sc.close();
}
}
```

Output:

Enter Regular Employee Name: Ram

Enter Base Salary: 30000

Enter Contract Employee Name: Ravi

Enter Base Salary: 20000

--- Salary Details ---

Regular Employee: Anitha

Base Salary: 30000.0

Total Salary (with 10% bonus): 33000.0

Contract Employee: Ravi

Total Salary: 20000.0

```
2 import java.util.Scanner;
3 abstract class Employee {
4     String name;
5     double baseSalary;
6
7     // Abstract method to calculate total salary
8     abstract void calculateSalary();
9 }
10
11 class RegularEmployee extends Employee {
12     double bonusRate = 0.1; // 10% bonus
13
14     void calculateSalary() {
15         double totalSalary = baseSalary + (baseSalary * bonusRate);
16         System.out.println("Regular Employee: " + name);
17         System.out.println("Base Salary: " + baseSalary);
18         System.out.println("Total Salary (with 10% bonus): " + totalSalary);
19     }
20 }
21
22 class ContractEmployee extends Employee {
23     void calculateSalary() {
24         System.out.println("Contract Employee: " + name);
25         System.out.println("Total Salary: " + baseSalary);
26     }
27 }
28
29 public class Main {
30     public static void main(String[] args) {
31         Scanner sc = new Scanner(System.in);
```

```

Scanner sc = new Scanner(System.in);

// Input for Regular Employee
System.out.print("Enter Regular Employee Name: ");
String regName = sc.nextLine();
System.out.print("Enter Base Salary: ");
double regSalary = sc.nextDouble();
sc.nextLine(); // Consume newline

// Input for Contract Employee
System.out.print("Enter Contract Employee Name: ");
String conName = sc.nextLine();
System.out.print("Enter Base Salary: ");
double conSalary = sc.nextDouble();

// Create objects
Employee e1 = new RegularEmployee();
e1.name = regName;
e1.baseSalary = regSalary;

Employee e2 = new ContractEmployee();
e2.name = conName;
e2.baseSalary = conSalary;

System.out.println("\n--- Salary Details ---");
e1.calculateSalary();
System.out.println();
e2.calculateSalary();

sc.close();
}

```

```

Enter Regular Employee Name: Ram
Enter Base Salary: 30000
Enter Contract Employee Name: Ravi
Enter Base Salary: 20000
|
--- Salary Details ---
Regular Employee: Ram
Base Salary: 30000.0
Total Salary (with 10% bonus): 33000.0

Contract Employee: Ravi
Total Salary: 20000.0

```

3. Banking System

Scenario:

A bank has different types of accounts: Savings and Current. Both accounts need a method to calculate interest, but the calculation differs for each account type.

Question:

Use an abstract class BankAccount with an abstract method calculateInterest() and implement it in SavingsAccount and CurrentAccount classes.

Code


```
abstract class BankAccount {  
    String accountHolder;  
    double balance;  
  
    BankAccount(String name, double bal) {  
        accountHolder = name;  
        balance = bal;  
    }  
  
    abstract void calculateInterest(); // Abstract method  
}  
  
class SavingsAccount extends BankAccount {  
    double interestRate = 0.04; // 4% interest  
  
    SavingsAccount(String name, double bal) {  
        super(name, bal);  
    }  
  
    void calculateInterest() {  
        double interest = balance * interestRate;  
        System.out.println("Savings Account Interest for " + accountHolder + " = " + interest);  
    }  
}  
  
class CurrentAccount extends BankAccount {  
    double interestRate = 0.02; // 2% interest
```

```
CurrentAccount(String name, double bal) {  
    super(name, bal);  
}  
  
void calculateInterest() {  
    double interest = balance * interestRate;  
    System.out.println("Current Account Interest for " + accountHolder + " = " + interest);  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        BankAccount acc1 = new SavingsAccount("Ram", 50000);  
        BankAccount acc2 = new CurrentAccount("Ravi", 80000);  
  
        acc1.calculateInterest();  
        acc2.calculateInterest();  
    }  
}
```

Output

Savings Account Interest for Ram = 2000.0

Current Account Interest for Ravi = 1600.0

```

2 abstract class BankAccount {
3     String accountHolder;
4     double balance;
5
6     BankAccount(String name, double bal) {
7         accountHolder = name;
8         balance = bal;
9     }
10
11     abstract void calculateInterest(); // Abstract method
12 }
13
14 class SavingsAccount extends BankAccount {
15     double interestRate = 0.04; // 4% interest
16
17     SavingsAccount(String name, double bal) {
18         super(name, bal);
19     }
20
21     void calculateInterest() {
22         double interest = balance * interestRate;
23         System.out.println("Savings Account Interest for " + accountHolder + " = " + interest);
24     }
25 }
26
27 class CurrentAccount extends BankAccount {
28     double interestRate = 0.02; // 2% interest
29
30     CurrentAccount(String name, double bal) {
31         super(name, bal);
32     }
33
34     void calculateInterest() {
35         double interest = balance * interestRate;
36         System.out.println("Current Account Interest for " + accountHolder + " = " + interest);
37     }
38 }
39
40 public class Main {
41     public static void main(String[] args) {
42         BankAccount acc1 = new SavingsAccount("Ram", 50000);
43         BankAccount acc2 = new CurrentAccount("Ravi", 80000);
44
45         acc1.calculateInterest();
46         acc2.calculateInterest();
47     }
48 }

```

```

Savings Account Interest for Ram = 2000.0
Current Account Interest for Ravi = 1600.0

```

POST LAB EXERCISE

- ✓ How is an abstract class different from a regular class?
- An abstract class is declared using the abstract keyword, whereas a regular class is declared normally.

- An abstract class cannot be instantiated, but a regular class can be instantiated.
- An abstract class may contain abstract methods without implementation, while a regular class contains only implemented methods.
- An abstract class is used as a base class for inheritance, whereas a regular class is used to create objects directly.

✓ Can you create an object of an abstract class? Why or why not?

- An abstract class is declared using the abstract keyword, whereas a regular class is declared normally.
- An abstract class cannot be instantiated, but a regular class can be instantiated.
- An abstract class may contain abstract methods without implementation, while a regular class contains only implemented methods.
- An abstract class is used as a base class for inheritance, whereas a regular class is used to create objects directly.

✓ What happens if a subclass does not implement an abstract method?

- If a subclass does not implement all the abstract methods of the parent class, it becomes incomplete.
- Such a subclass must be declared as abstract.
- If it is not declared abstract, the compiler will generate an error.
- This rule ensures that all abstract methods are implemented before creating objects.

- ✓ Can an abstract class exist without any abstract methods?
 - Yes, an abstract class can exist without any abstract methods.
 - It can contain only concrete (implemented) methods.
 - Declaring a class as abstract prevents object creation of that class.
 - It is mainly used to provide common functionality to derived classes.

- ✓ Can an abstract class extend another abstract class?
 - Yes, an abstract class can exist without any abstract methods.
 - It can contain only concrete (implemented) methods.
 - Declaring a class as abstract prevents object creation of that class.
 - It is mainly used to provide common functionality to derived classes.

Result:

Thus the abstract classes and methods were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	

Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		